# Laboratório de Concorrência · Aula 10

Felipe de Almeida Matrone · RA 24051005-2 · Turma A
Gerado em 09/10/2026, 20:33:40 · tempo registrado no cronômetro: 0:00

## Preparação: Concorrência em cinco minutos

**0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.**

A sincronização cooperativa ocorre quando uma pessoa necessita esperar a outra terminar alguma coisa. Por exemplo, esperar alguém terminar de usar o banheiro para poder entrar. Por outro lado, a sincronização de competição acontece quando duas pessoas querem usar um recurso idêntico simultaneamente, como duas pessoas querendo usar o mesmo computador, Como exemplo.

## Prática 1: A condição de corrida

**Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?**

Minha expectativa é que o programa mostre 2.000.000, pois cada thread vai somar 1 ao contador um milhão de vezes.

**1ª execução (n = 1.000.000)**

1014840

**2ª execução (n = 1.000.000)**

1357727

**3ª execução (n = 1.000.000)**

1026576

**Saída do programa**

*não respondido*




**1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?**

Não. Além disso, os valores foram distintos nas três execuções. Isso ocorre pois as duas threads acessam a variável compartilhada ao mesmo tempo, fazendo com que algumas atualizações sejam perdidas. Como a ordem de execução muda, o resultado também pode variar a cada execução, como foi o caso.

### 1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A situação ocorre quando as duas threads leem o valor idêntico do contador antes de gravar. Como exemplo, se o contador vale 10, a thread 1 lê 10 e soma 1, chegando a 11. Antes de gravar, a thread 2 também lê 10 e soma 1. As duas gravam 11, então uma atualização se perde e o contador fica com 11 em vez de 12.

### 1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, o valor obtido pode ter sido o esperado, pois, com menos repetições, torna-se mais complexo acontecer uma condição de corrida. Entretanto, também pode ser que ocorram resultados diferentes também.

## Prática 2: Todas as respostas possíveis (Figura 13.1)

### Ordens testadas no simulador

AAABBB → 8

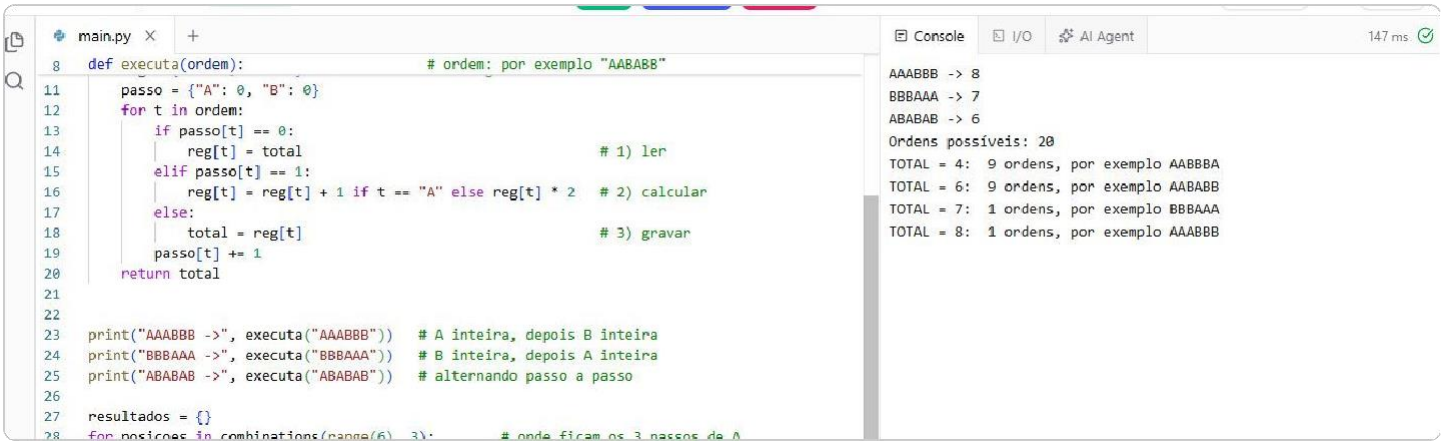
BBBAAA → 7

BABABA → 4

AABBAB → 6

### Saída do programa

*não respondido*




### 2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis de execução e 4 resultados diferentes: 4, 5, 7 e 8. Dessas ordens, 6 produzem o valor obtido correto, que é 8.

### 2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Uma ordem que dá 4 é BAAABB. Primeiro, B lê o valor 3 e calcula 6, No entanto ainda não grava. Depois, A lê 3 e calcula 4. B grava 6, No entanto A grava 4 no final, trocando o valor de B. Dessa forma, o valor obtido final é 4, pois a atualização de B foi sobrescrita pela de A.

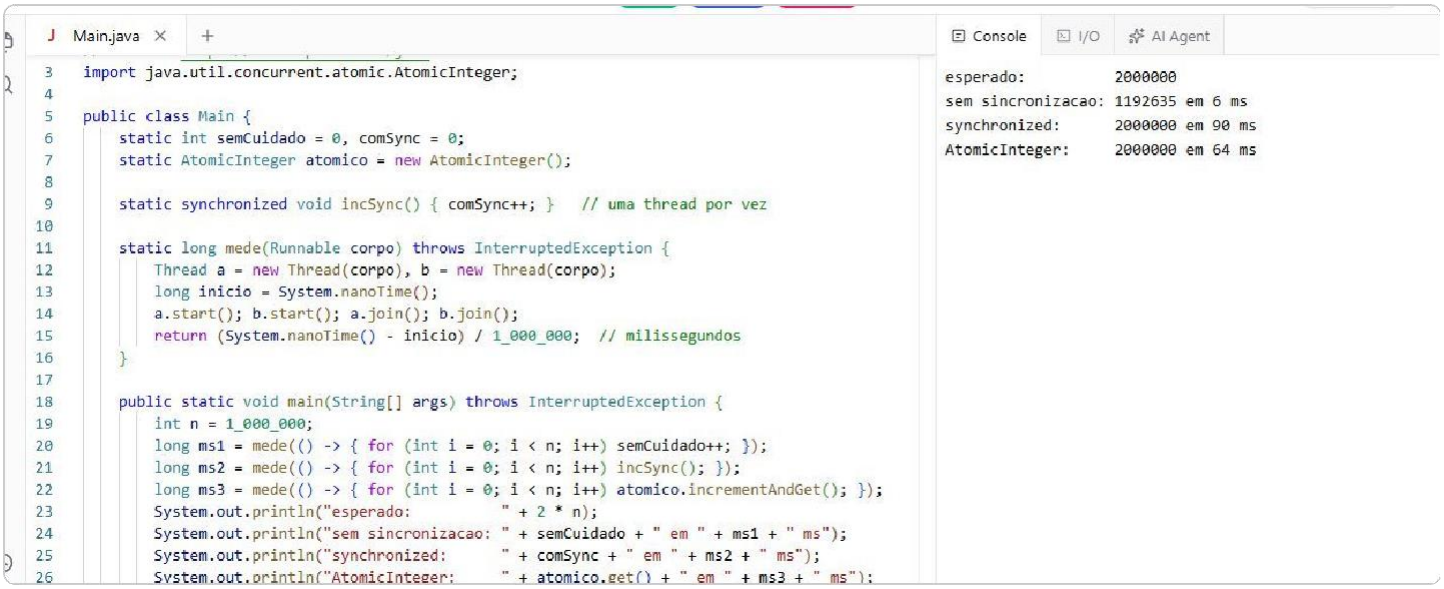
# Prática 3: Corrigindo a corrida

**Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?**

Considero que a versão sem sincronização será a mais rápida, pois não necessita controlar o acesso à variável. Entretanto, ela pode produzir um resultado incorreto. Já as versões com synchronized e AtomicInteger devem apresentar o valor obtido correto, pois evitam a perda de atualizações.

**Saída do programa**

*não respondido*




```java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

    static synchronized void incSync() { comSync++; }   // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000;  // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado:          " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger:     " + atomico.get() + " em " + ms3 + " ms");
```

**3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?**

As versões com synchronized e AtomicInteger deram o valor correto de 2.000.000, enquanto a versão sem sincronização pode produzir um valor menor. As versões corretas costumam ser mais lentas pois necessitam controlar o acesso à variável compartilhada para impedir que as threads percam atualizações.

**3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?**

Não, pois cada AtomicInteger garante a atualização de um saldo individualmente, No entanto não assegura que a transferência inteira seja concluída de modo correto. Seria preciso sincronizar as duas operações para assegurar que o dinheiro seja retirado de uma conta e colocado na outra sem interferência de outras transações.

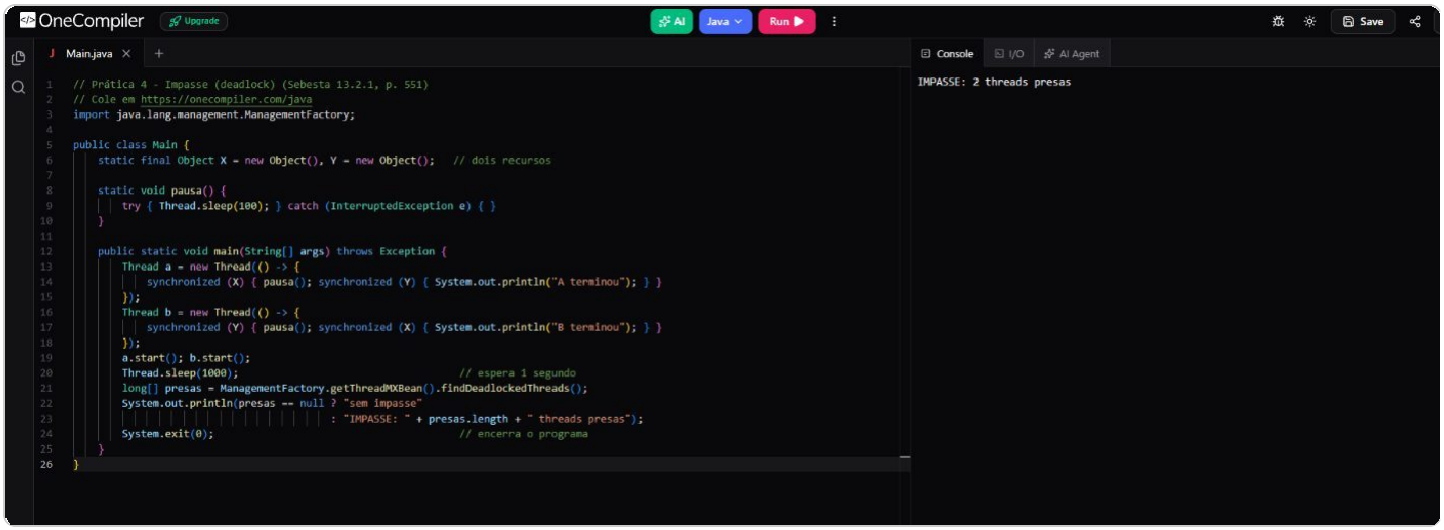
# Prática 4: Impasse (deadlock)

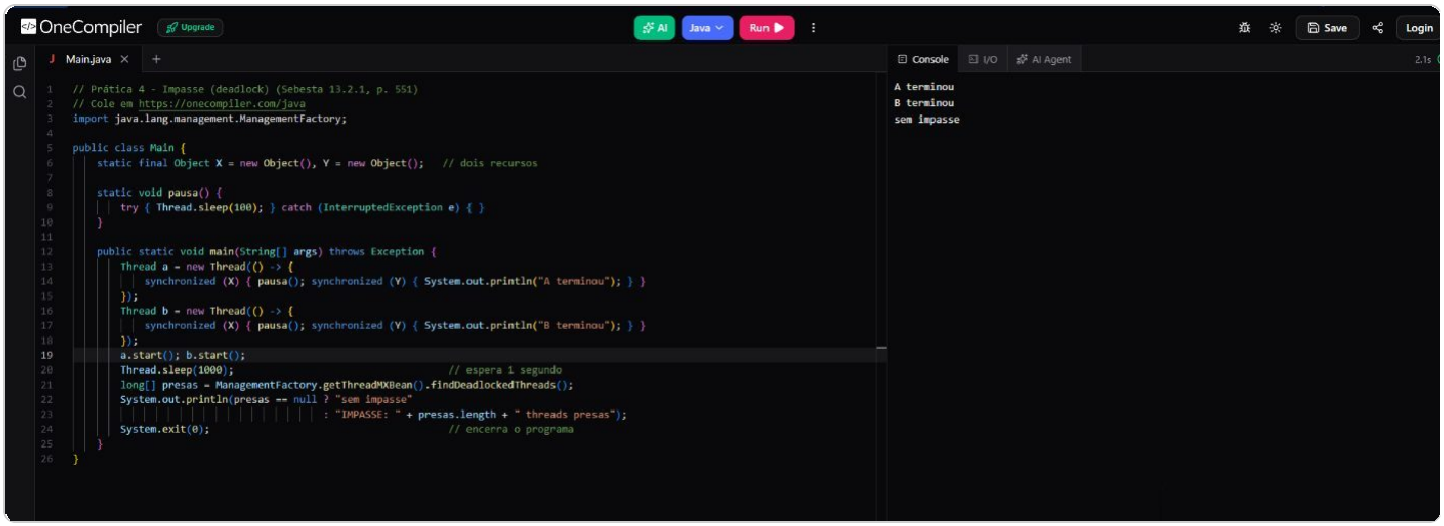
**Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?**

Não, nunca irão aparecer. A função pausa() garante que cada thread pegue um recurso e demore 100 ms para tentar obter o segundo, Desse modo criado um deadlock.

**Saídas (antes e depois da correção)**

*não respondido*




### 4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

Thread A obtém para utilizar o recurso X simultaneamente que a Thread B obtém para utilizar o recurso Y. Depois, A tenta obter Y, No entanto não consegue, pois está sendo executado por B então, a thread A espera B finalizar o uso do recurso Y. Simultaneamente, B tenta obter X, No entanto não consegue, por conta de estar sendo utilizado por X então, fica esperando que A libere o recurso X.

### 4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Com as duas threads solicitando os mesmos recursos na mesma ordem, a espera circular deixaria de ocorrer. Desse modo, A thread A iria obter o recurso X, então a thread B precisaria esperar A terminar para começar a executar. Então, a thread A iria usar X e depois Y e liberar os dois recursos para que B começasse a sua execução.

## Prática 5: Mensagens em vez de memória compartilhada

### Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

No total demoraria 200 ms para que 3 trabalhadores concluam as 6 tarefas.

Com 1 trabalhador demoraria 600ms

com 600 trabalhadores demoraria 100ms

### Tempo com 1 trabalhador

600

### Tempo com 3 trabalhadores

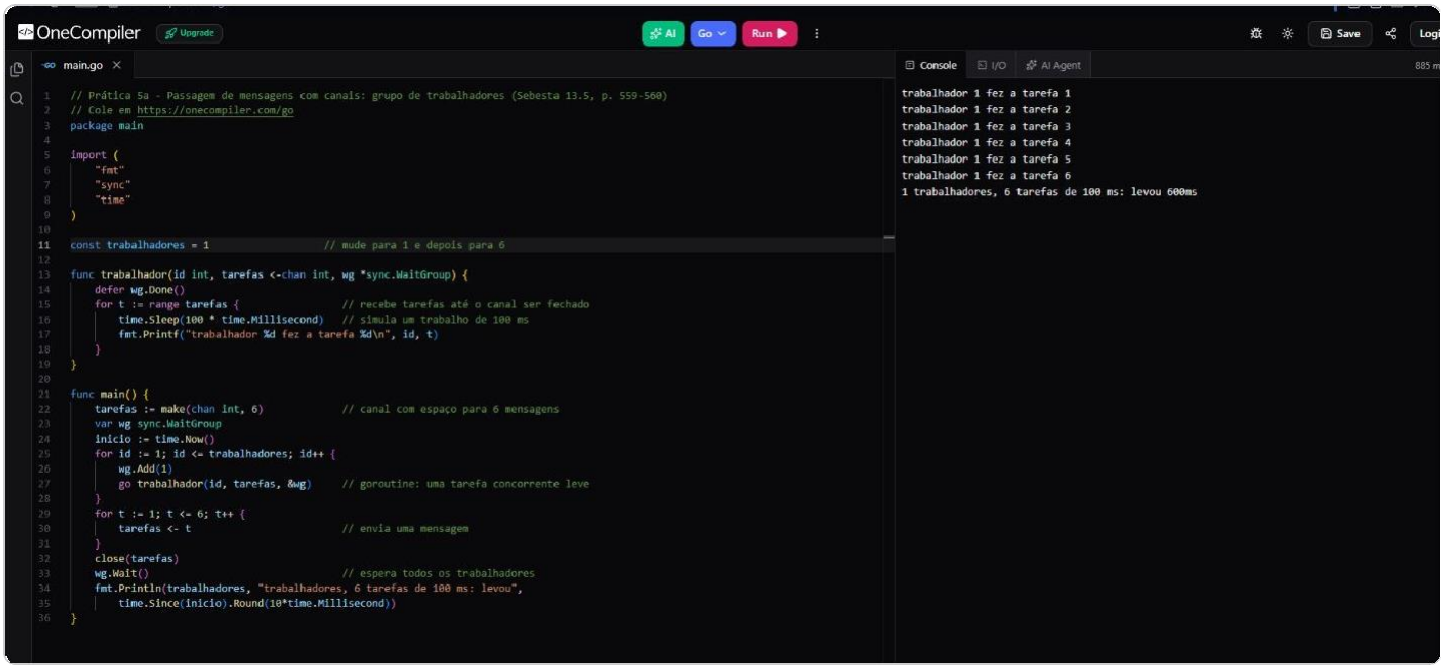
200

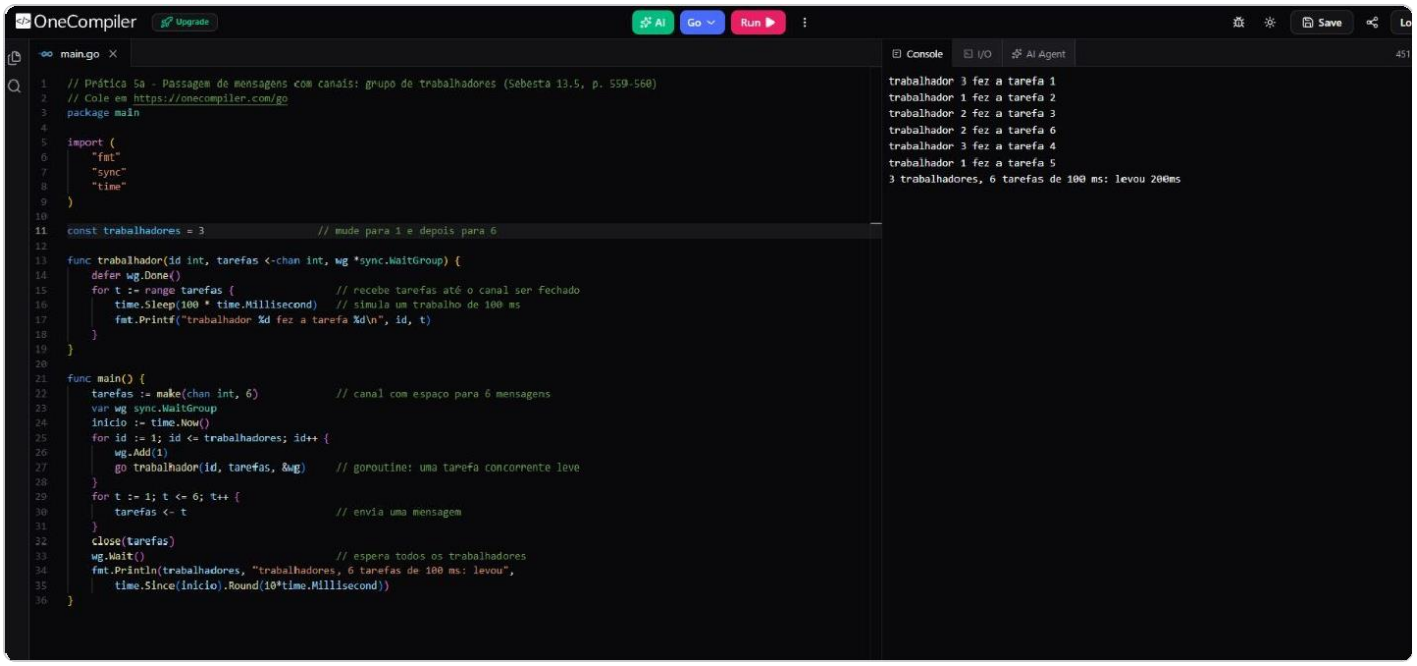
### Tempo com 6 trabalhadores

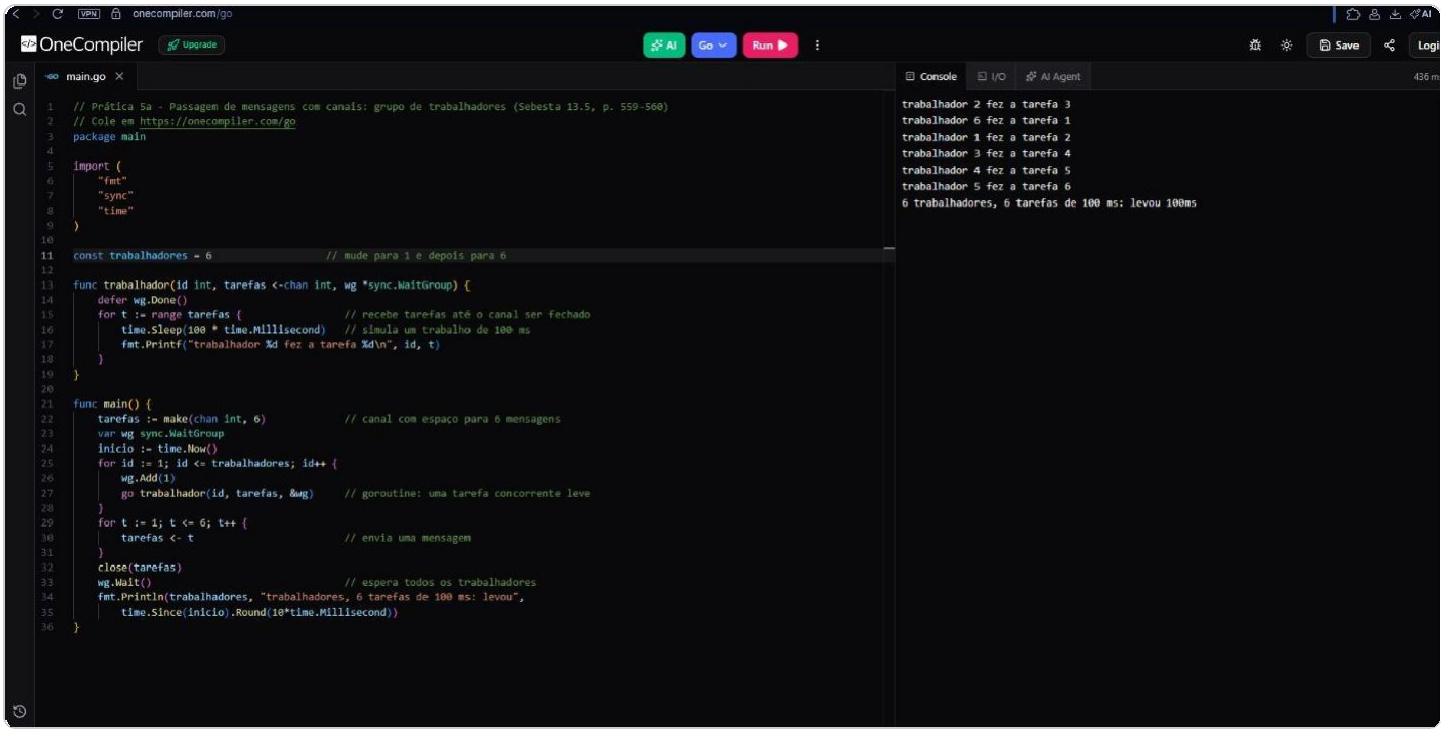
100

### Saída do programa

*não respondido*



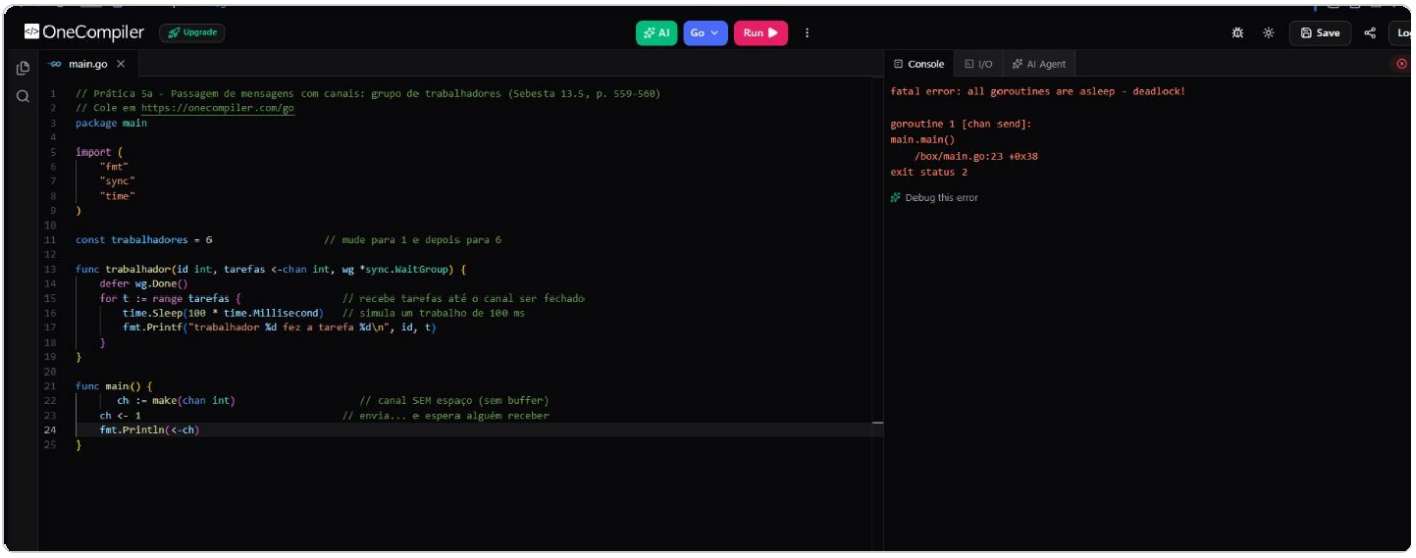


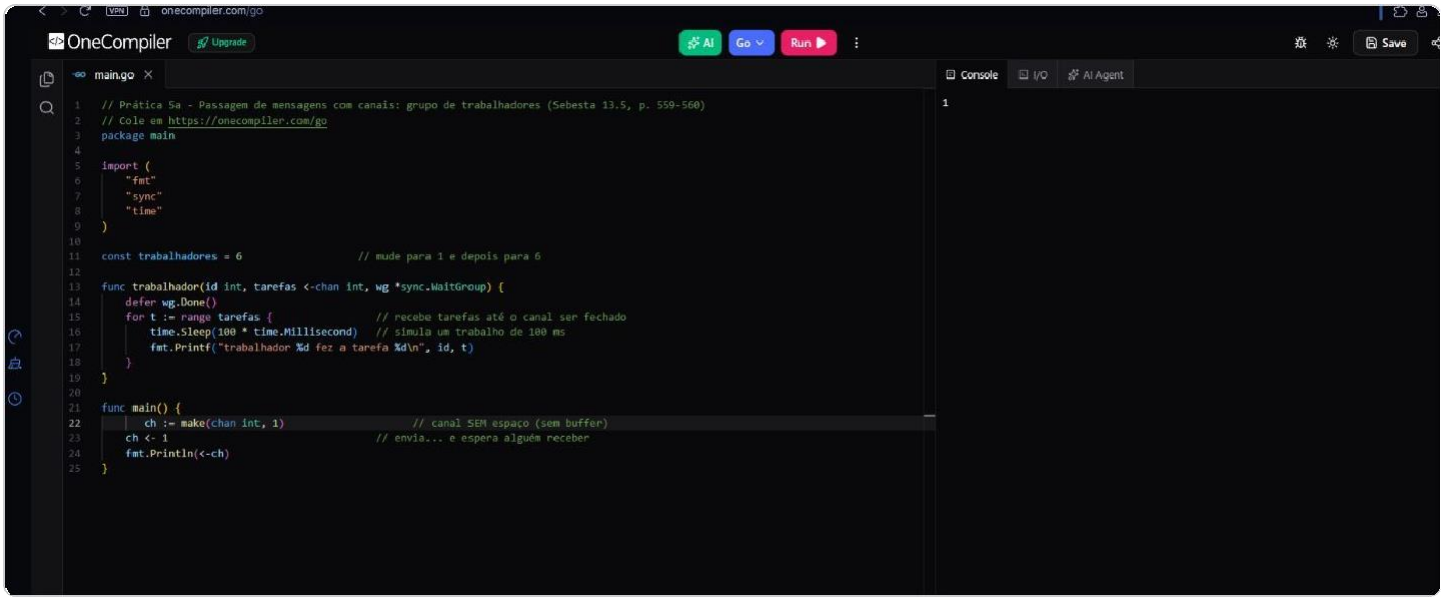

### 5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Pois, nesse sistema, cada trabalhador seria independente do outro trabalhador. Desse modo, quando um trabalhador conclui uma tarefa, ele já pegaria a próxima disponível, não sendo necessário esperar a comunicação com outro trabalhador. Não necessita de travas pois nada é compartilhado e modificado pelas rotinas, cada tarefa é uma mensagem para que a comunicação ocorra

### Saídas (antes e depois da troca)

*não respondido*






**5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)**

O primeiro programa entra em deadlock, basicamente pois o canal criado com make(chan int) não tem capacidade de armazenamento para mensagens e consequentemente, o emissor necessita esperar que outra goroutine esteja pronta para receber. Nesse código, existe apenas a goroutine main ela fica bloqueada na linha ch <- 1 aguardando um receptor, embora a operação <-ch destinada a receber a mensagem esteja logo na próxima linha — uma linha que jamais é executada. Dessa forma, a main aguarda uma ação que apenas ela mesma poderia realizar posteriormente; o Go detecta que todas as goroutines estão bloqueadas e encerra o programa, relatando um erro de deadlock. Com make(chan int, 1), o canal adquire capacidade para uma mensagem portanto, a operação de envio simplesmente armazena o valor nesse espaço e continua sem esperar. Depois, <-ch recupera o valor do canal e o programa imprime 1

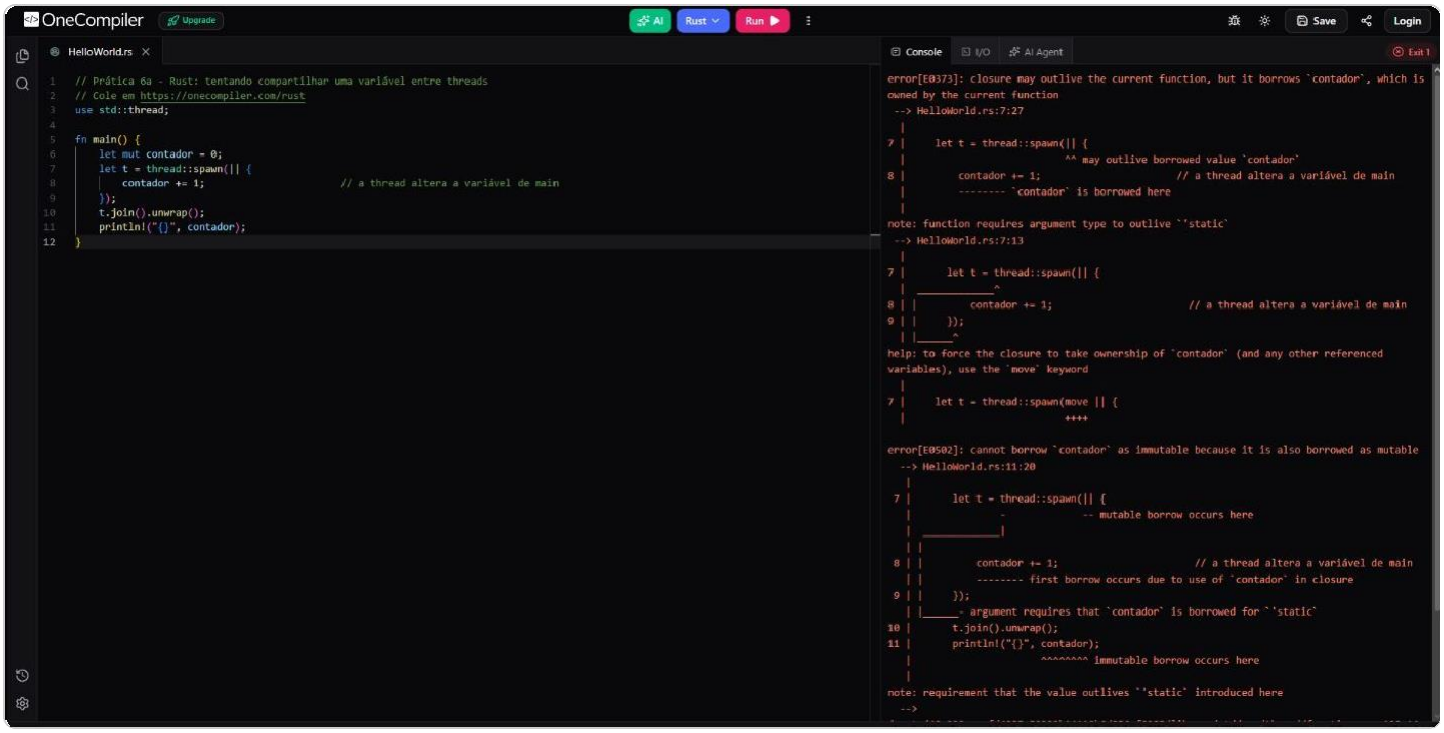
## Prática 6: O compilador como guarda

**Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?**

o compilador impede rodar, pois a thread pode continuar viva depois que main terminar e a variável contador pertence a main.

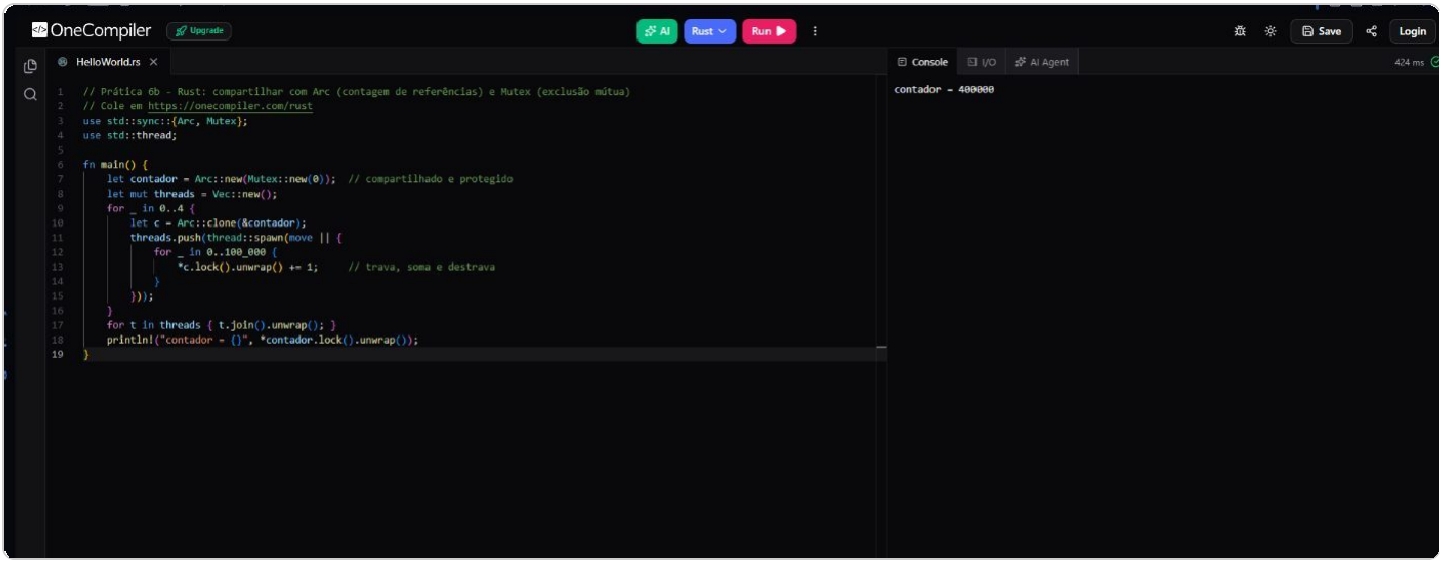
**Saídas (o erro e o resultado com move)**

*não respondido*




**Saída do programa**

*não respondido*




### 6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, uma thread recebe uma cópia do contador e o incrementa em 1, No entanto o valor inicial em main continua igual a 0. É por isso que 0 é exibido na impressão. O programa compila, No entanto o valor obtido não é o esperado, pois cada uma tem o seu próprio número.

### 6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, a corrida só foi identificada quando o programa rodou, pois o código compilava normalmente e o valor final saía errado, mudando de uma execução para outra, o que torna mais difícil localizar o problema. Por outro lado, em Rust, o erro foi descoberto antes da execução, na compilação, pois o compilador impede a thread alterar uma variável sem proteção. Desse modo, esse tipo de erro não chega até o usuário, e para compartilhar o dado o programador

necessita usar Arc junto com Mutex. A desvantagem é que o código torna-se mais complexo de escrever, pois é preciso entender as regras do compilador e seguir o que ele exige para o programa compilar.

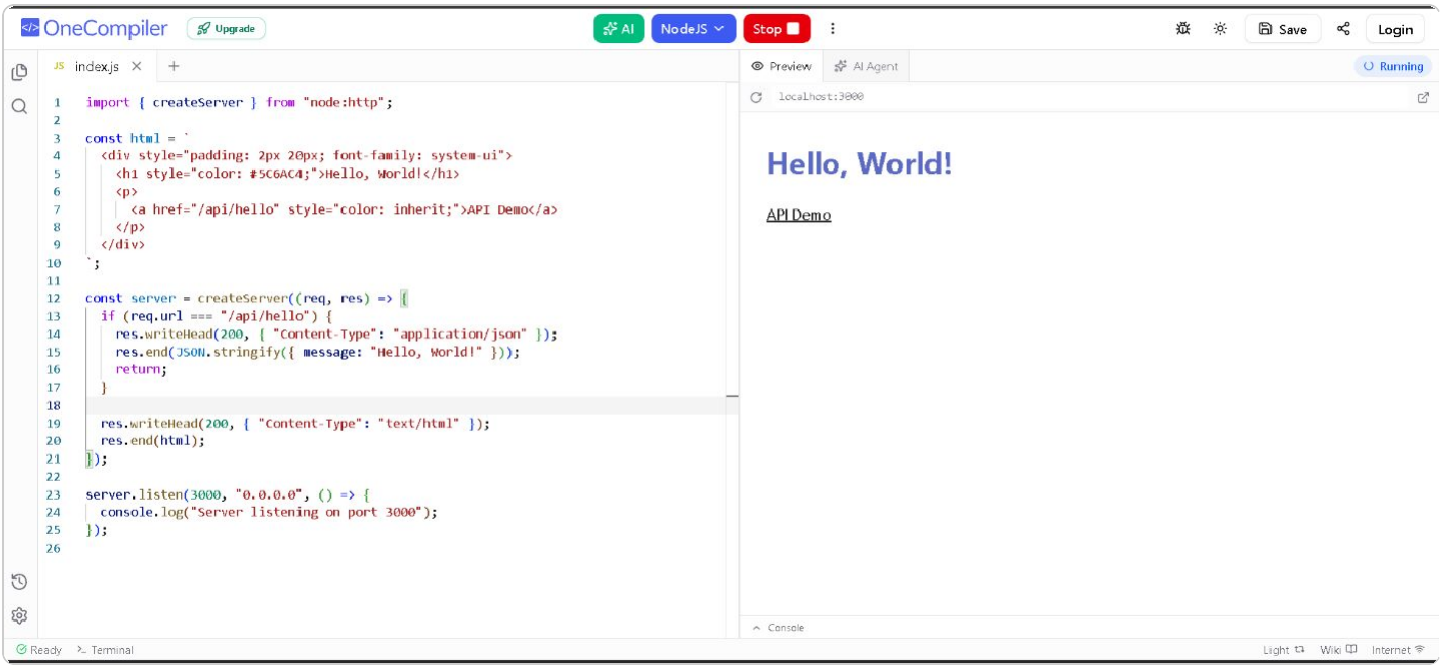
## Prática 7: Uma thread só: o laço de eventos

### Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B

### Saída do programa

*não respondido*




```
import { createServer } from "node:http";

const html = `
  <div style="padding: 2px 20px; font-family: system-ui">
    <h1 style="color: #5C6AC4;">Hello, World!</h1>
    <p>
      <a href="/api/hello" style="color: inherit;">API Demo</a>
    </p>
  </div>
`;

const server = createServer((req, res) => {
  if (req.url === "/api/hello") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ message: "Hello, World!" }));
    return;
  }

  res.writeHead(200, { "Content-Type": "text/html" });
  res.end(html);
});

server.listen(3000, "0.0.0.0", () => {
  console.log("Server listening on port 3000");
});
```

### 7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem de saída é: A, D, C, B. Esse comportamento ocorre pois o JavaScript executa todo o código síncrono em primeiro lugar, imprimindo as letras A e D. Logo depois, o laço de eventos processa as "microtarefas" pendentes, como é o caso das Promises, executando a função que imprime o C. Somente ao final é que o sistema processa as tarefas agendadas ("macrotarefas"), como o setTimeout, pelo que o B é impresso por último, mesmo que o seu tempo de espera tenha sido definido como 0 milissegundos.

### 7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node. js que atende muitos usuários e faz um cálculo pesado?

O temporizador sofreu atraso pois o JavaScript e o Node.js utilizam uma única thread com um laço de eventos (event loop). A função ocupado(300) monopolizou e bloqueou essa única thread durante cerca de 300 milissegundos, impossibilitando que o laço de eventos avançasse para libertar e processar o temporizador que já estava pronto. Num servidor em Node.js, isto tem um efeito significativo: um único cálculo pesado irá "prender" a thread principal, fazendo com que o servidor congele e fique incapaz de responder a todos os outros utilizadores até que o cálculo termine.

## Fechamento: Quem impede a corrida?

### Java sem sincronização

Nenhum mecanismo: ocorre uma condição de corrida

### Java com synchronized

O desenvolvedor, por meio da sincronização

### Go com canais

A arquitetura, pois não existe memória compartilhada

### Rust com Arc e Mutex

A verificação realizada pelo compilador

### JavaScript

A existência de apenas uma thread

### 8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

O problema que pode acontecer é uma clássica "condição de corrida", em razão da falta de sincronização de competição. Ambas as threads conseguem ler o valor "1 lugar disponível" simultaneamente (antes de qualquer uma gravar a atualização), processando ambas a venda e atribuindo o mesmo bilhete a duas pessoas diferentes. Para impedir isto, usaria mecanismos de exclusão mútua: um monitor genérico (como o synchronized em Java) para assegurar que apenas um pedido acede ao inventário por vez, operações atómicas no próprio dado, ou uma arquitetura baseada na passagem de mensagens (como os canais em Go).

### 8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que realmente me surpreendeu, foi a linha de aprendizado. Além disso esse Html muito bom realizado pelo professor.